

The optional argument *dim* is used to specify the dimension of the matrix along which the mean is to be calculated. By default, the dimension is set to 1. You may also specify the dimension to be a maximum of 2 for a two-dimensional matrix. When you specify *dim* = 2, the mean is calculated along the rows. So the mean is a column vector. For example:

```
octave -3.2.2.58 > x=[1 8 3 9 6]
x =
    1    8    3
    4    9    6

octave -3.2.2.59 > mean(x, 2)
ans =

4.0000
6.3333
```

If you have a higher-dimensional matrix, you may specify *dim* accordingly.

- **median:** The syntax and semantics of the *median* function are similar to the *mean* function: *median(x, dim)*, where *dim* has a similar interpretation as above.
- **kurtosis:** *kurtosis* is the degree of how much a probability distribution peaks. In Octave, it can be found by using the *kurtosis* function. The syntax is similar to the *median* function introduced above.
- **statistics:** If you want most of the descriptive statistics on your data, use the *statistics* function. If *x* is a matrix, it returns a matrix with the minimum, first quartile, median, third quartile, maximum, mean, standard deviation, extent of being skewed and kurtosis of the columns of *x* as its rows. If *x* is a vector, it is treated as a column vector.

Besides the above, you also have functions for calculating the *covariance*, *correlation*, *extent of being skewed* and *moment* of a set of multi-dimensional data.

Basic statistical functions

- **Mahalanobis distance** is the distance measure that is useful for identifying the similarity of

an unknown sample of data to a known one. In Octave, you can use the function *mahalanobis(x,y)* to find the Mahalanobis' D-square distance between the multivariate samples *x* and *y*, which must have the same number of components (columns), but may have a different number of observations (rows).

- **Miscellaneous functions:** The *perms* function may be used to generate all permutations of a matrix or a vector, one row at a time. *values* is a useful function that returns the different values in a row or a column vector in ascending order—i.e., it does not return the duplicates. Please note that values do not work on a matrix. There are also other functions of interest to a student of statistics, such as *logit* and *probit*.

Other features: Besides the rather basic features described earlier, there is extensive support for performing several different statistical tests such as equal mean values, equal median values and others. A large number of probability distributions is available, using which you can generate random numbers and matrices.

Please look up the Octave reference manual under the statistics section for an extensive index of the functions available for the various statistical analyses of your numerical data.

Polynomial manipulations

Let us now learn to work with univariate polynomials in Octave. A polynomial is represented by a vector containing its coefficients. For example, a polynomial such as $p(x) = 2x^4 + 3x^2 + 4x + 5$ will be represented in Octave by the vector *c* = [2 3 4 5]. The *polyval* command prints the formatted polynomial from a coefficient vector. Let's try this out:

```
octave -3.2.2.92 > c=[2 3 4 5];
octave -3.2.2.93 > polyval(c, 'x')
```

```
25x3 + 38x2 + 48x1 + 5
```

```
octave -3.2.2.94 > polyval(c, 's')
25s3 + 38s2 + 48s1 + 5
```

As you can see, we have first defined the coefficient vector *c*, and then used the *polyval* function to print the formatted polynomial. The second argument to *polyval* is the variable name in the univariate polynomial.

Roots and evaluation

The *roots* function returns the roots of the polynomial, given its coefficient vector, *c*:

```
octave -3.2.2.100 > r=roots(c)
r =

-1.37113 + 0.00000i
-0.09443 + 1.34876i
-0.09443 - 1.34876i
```

Now let's look at how good the roots are. Let us evaluate the polynomial at the obtained roots:

```
octave -3.2.2.101 > polyval(c, r (1))
ans = -2.6645e-15
octave -3.2.2.102 > polyval(c, r (2))
ans = 6.2172e-15 + 3.6915e-15i
octave -3.2.2.103 > polyval(c, r (3))
ans = 6.2172e-15 - 3.6915e-15i
```

Okay, so it looks like the roots are pretty good, as the value of the polynomial is tending to 0 at those points. The *polyval* takes the coefficient vector as the first argument and the point at which the value is to be evaluated as the second argument. Instead of passing the roots one by one, as we have done, we could also send the whole matrix to *polyval*, as follows:

```
octave -3.2.2.104 > polyval(c, r)
ans =

-2.6645e-15 + 0.0000e+00i
5.8842e-16 + 3.9527e-15i
6.2172e-15 - 3.6915e-15i
```

As a general rule, when a matrix or a vector is passed as the second